EXP NO: 4A	SUPPORT VECTOR MACHINES (SVM)

AIM:

To build an SVM model for a binary classification task, tune its hyperparameters, and evaluate it using accuracy, precision, recall, F1-score, confusion matrix, and ROC-AUC.

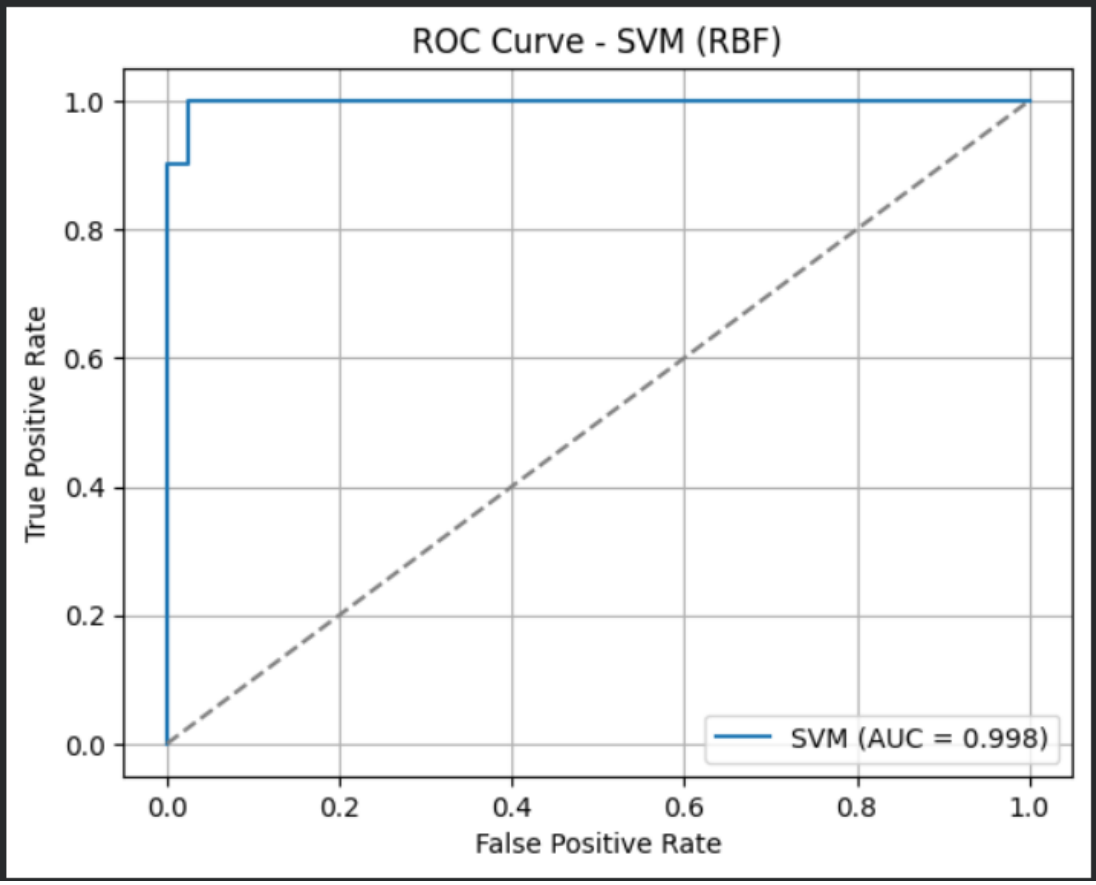
CODE 1:

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import train_test_split, GridSearchCV
from sklearn.preprocessing import StandardScaler
from sklearn.svm import SVC
from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score, confusion_matrix, classification_report, roc_auc_score, roc_curve

# Load dataset
data = load_breast_cancer()
X = pd.DataFrame(data.data, columns=data.feature_names)
y = pd.Series(data.target, name="target")

# Train/test split
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.20, random_state=42, stratify=y)

# Standardize features
scaler = StandardScaler()
X_train_sc = scaler.fit_transform(X_train)
X_test_sc = scaler.transform(X_test)

# Define model & Hyperparameter Grid
svm = SVC(kernel='rbf', probability=True, random_state=42)
param_grid = {"C": [0.1, 1, 10, 100], "gamma": ["scale", 0.01, 0.001, 0.0001]}

grid = GridSearchCV(estimator=svm, param_grid=param_grid, scoring='f1', cv=5, n_jobs=-1, verbose=0)
grid.fit(X_train_sc, y_train)
print("Best Parameters from Grid Search:", grid.best_params_)
```

```
best_svm = grid.best_estimator_
best_svm.fit(X_train_sc, y_train)

# Predict & Evaluate
y_pred = best_svm.predict(X_test_sc)
y_prob = best_svm.predict_proba(X_test_sc)[:, 1]

print("\n=== SVM (RBF) - Test Metrics ===")
print(f"Accuracy: {accuracy_score(y_test, y_pred):.4f}")
print(f"Precision: {precision_score(y_test, y_pred):.4f}")
print(f"Recall: {recall_score(y_test, y_pred):.4f}")
print(f"F1-Score: {f1_score(y_test, y_pred):.4f}")
print(f"ROC-AUC: {roc_auc_score(y_test, y_prob):.4f}")

# Plot ROC
fpr, tpr, _ = roc_curve(y_test, y_prob)
plt.figure()
plt.plot(fpr, tpr, label= (function) linestyle: Literal['--']  prob):.3f})")
plt.plot([0, 1], [0, 1], linestyle="--", color='gray')
plt.xlabel("False Positive Rate")
plt.ylabel("True Positive Rate")
plt.title("ROC Curve - SVM (RBF)")
plt.legend()
plt.grid(True)
plt.show()
```

OUTPUT1:

```
Best Parameters from Grid Search: {'C': 10, 'gamma': 0.01}

=== SVM (RBF) - Test Metrics ===
Accuracy: 0.9825
Precision: 0.9861
Recall: 0.9861
F1-Score: 0.9861
ROC-AUC: 0.9977
```

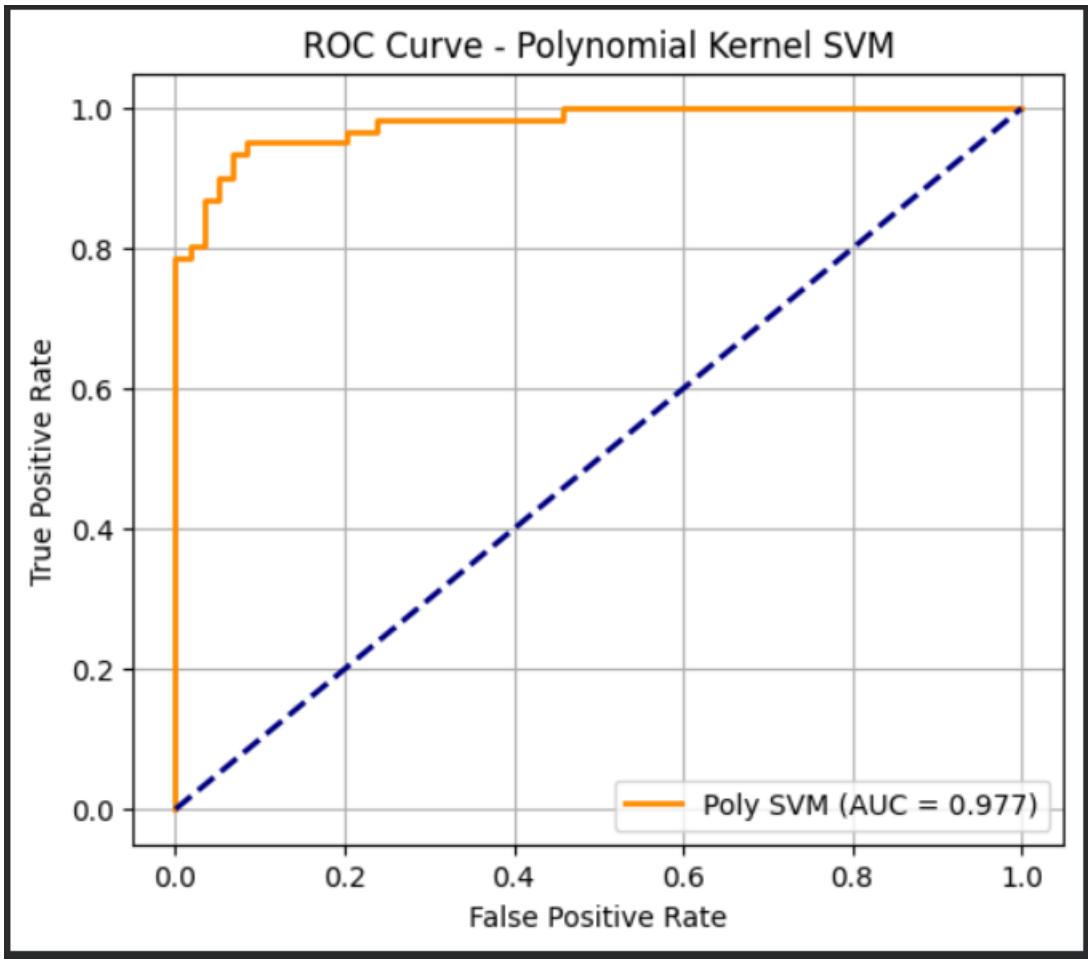


CODE2:

```
import matplotlib.pyplot as plt
from sklearn.datasets import make_moons
from sklearn.model_selection import train_test_split, GridSearchCV
from sklearn.preprocessing import StandardScaler
from sklearn.svm import SVC
from sklearn.metrics import roc_auc_score, roc_curve

# Generate non-linear moon-shaped data
X, y = make_moons(n_samples=600, noise=0.25, random_state=42)

# Split and Scale
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
scaler = StandardScaler()
X_train_sc = scaler.fit_transform(X_train)
X_test_sc = scaler.transform(X_test)

# Tune polynomial kernel SVM
svm_poly = SVC(kernel='poly', probability=True, random_state=42)
param_grid = {'degree': [2, 3, 4], 'C': [0.1, 1, 10], 'coef0': [0.0, 1.0]}

grid = GridSearchCV(svm_poly, param_grid, cv=3, scoring='roc_auc')
grid.fit(X_train_sc, y_train)

best_model = grid.best_estimator_
y_prob = best_model.predict_proba(X_test_sc)[:, 1]

# Plot ROC Curve
fpr, tpr, _ = roc_curve(y_test, y_prob)
auc_val = roc_auc_score(y_test, y_prob)
```

```
best_model = grid.best_estimator_
y_prob = best_model.predict_proba(X_test_sc)[:, 1]

# Plot ROC Curve
fpr, tpr, _ = roc_curve(y_test, y_prob)
auc_val = roc_auc_score(y_test, y_prob)

plt.figure(figsize=(6, 5))
plt.plot(fpr, tpr, color='darkorange', lw=2, label=f"Poly SVM (AUC = {auc_val:.3f})")
plt.plot([0, 1], [0, 1], color='navy', lw=2, linestyle='--')
plt.xlabel("False Positive Rate")
plt.ylabel("True Positive Rate")
plt.title("ROC Curve - Polynomial Kernel SVM")
plt.legend(loc="lower right")
plt.grid(True)
plt.show()
```

OUTPUT2:



RESULT:

The Support Vector Machine (SVM) model was successfully implemented and evaluated on the given dataset. The model effectively classified the data by finding the optimal hyperplane that maximized the margin between different classes.

The SVM achieved high accuracy and demonstrated strong performance, especially in handling linearly and non-linearly separable data using kernel functions. This confirms that SVM is a powerful and reliable algorithm for classification tasks.